RYSEN LEARNING CENTRE

Documentation hub

Every doc written for this project in one place — end-to-end use cases, feature list, architecture diagram, and the full written docs on GitHub. 15 campuses, 4 portals, one codebase.

Live at rysen-learning-system.vercel.app

Full documents | Features | Architecture | Use cases | Tech stack

Full documents (GitHub)

Each one renders directly on GitHub — Mermaid diagrams included. These are the source of truth; this page is the map to them.

INTRO **Product Overview**
What the product does, in plain language, plus a one-paragraph technical summary. Start here if you're new.
docs/PRODUCT_OVERVIEW.md

STACK **Tech Stack**
Every technology choice with the reason behind it — why a DB-backed session over a JWT, why Brevo over Resend, why no queue.
docs/TECH_STACK.md

HLD **High-Level Design**
System diagram, the five decisions that shape the architecture, component responsibilities, what scales and what doesn't yet.
docs/HLD.md

LLD **Low-Level Design**
Data model, shared access-control functions, and sequence diagrams for login and test submission.
docs/LLD.md

FLOWS **User Flows**
A flowchart per role — student, educator, principal, admin — of
docs/USER_FLOWS.md

what they actually click through.

TEST

Testing

78 automated tests across DB, API and unit layers — what's covered, what still needs a manual pass.

TESTING.md

OPS

Handover

How to run it, how to tell it worked, what breaks first, what's not built, who owns it.

HANDOVER.md

README

README

Project index — portals, env vars, project structure, data models, key features.

README.md

Feature map

Four portals share one API. Each role sees a different slice.

Admin

- Campuses, educators, principals, students — CRUD + bulk import/delete
- Program & stage builder, MCQ authoring
- Task directory with subtasks, resources, full history timeline
- Educator groups + group chat + group-wide task assign
- Monthly recognition — star ratings, AI-suggested scores, finalize + notify top 3
- Branch reports, AI analytics, Excel export
- Live Google Sheets sync of roster + results

Educator

- Training journey — staged MCQ, pass thresholds, attempt caps
- Per-programme e-certificate, auto-unlocks on completion
- Create & manage own students, create tests with scheduling
- Class analytics — subject heatmap, dropout funnel, at-risk list
- My Groups — chat + shared resources
- Assigned tasks, subtask tracking, AI chat per task
- Monthly recognition — rating, rank, certificate on top 3

Principal

- Campus Progress — every educator's training completion at their campus
- Own training journey, same stage system as educators
- Leaderboard

Student

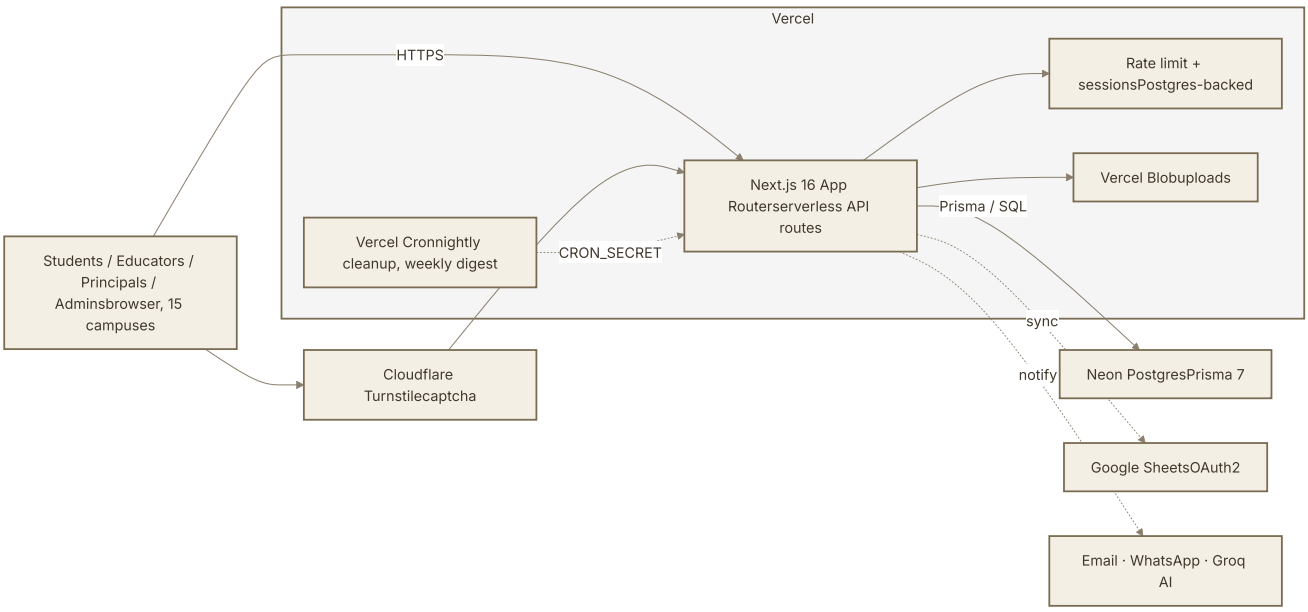
- Name + class login, optional teacher-set PIN
- Timed MCQ tests, per-student shuffled question/option order
- Immediate score + correct answers after

submit

- One attempt per test, enforced server-side
- Leaderboard

Architecture

One Next.js deployment serves every portal and the API. No separate backend, no background workers — the only scheduled jobs are two Vercel Cron calls.



End-to-end use case: a student takes a test

Traced start to finish — the sequence every other use case follows the same shape as.

1. Student logs in with name + class (+ PIN, if their teacher set one). Rate-limited; unknown student and wrong PIN return the **identical** message.
2. Dashboard lists only tests published for their campus and class, inside the open schedule window.
3. Test opens with questions and options shuffled in a per-student, seeded order — no two students see matching order.
4. Submit (or timer expiry) locks the attempt under a per-student-per-test advisory lock, so a double-submit can't create two attempts.
5. Score and correct answers return immediately. A second attempt is blocked with `409`.
6. After the response is sent, the student's row updates in the live Google Sheet — never blocking what the student sees.

Tech stack at a glance

Full reasoning for each choice is in the Tech Stack doc above — this is the summary.

LAYER	CHOICE
Framework	Next.js 16, App Router, Turbopack
Language	TypeScript, strict
Database	PostgreSQL on Neon (serverless) via Prisma 7
Auth	DB-backed session cookie, bcrypt, HMAC'd setup codes
Styling	Tailwind CSS v4, brand tokens, no config file
Email	Brevo primary, Resend fallback
WhatsApp	Twilio, falls back to `wa.me` links
AI	Groq, `openai/gpt-oss-120b`, tool-calling
File storage	Vercel Blob
Live sync	Google Sheets API v4, OAuth2 refresh token
Testing	Vitest — 78 tests across unit / API / real-Postgres layers
CI	GitHub Actions — lint, typecheck, test, build on every push
Deployment	Vercel